

Topic: Complex Retrieval Queries using GROUP BY

Q. Explain the use of the GROUP BY clause in complex queries. Differentiate between WHERE and HAVING, and demonstrate the order of execution when combining GROUP BY, HAVING, and ORDER BY.

📌 Definition to Remember

The **GROUP BY** clause groups rows with identical values in specified columns into summary rows, allowing **Aggregate Functions** (COUNT, SUM, AVG, MAX, MIN) to be applied to each group separately. The **HAVING** clause filters these groups — similar to WHERE but applied *after* grouping.

1. GROUP BY with Aggregate Functions

Instead of calculating an aggregate for the entire table, `GROUP BY` calculates it **per unique group**.

```
-- Find the total number of employees in each department
SELECT Department, COUNT(Emp_ID) AS Total_Employees
FROM Employee
GROUP BY Department;
```

Result: One summary row per department.

2. HAVING Clause

`WHERE` cannot use aggregate functions. `HAVING` was introduced specifically to **filter groups** after `GROUP BY` executes.

```
-- Find departments with more than 5 employees
SELECT Department, COUNT(Emp_ID)
FROM Employee
GROUP BY Department
HAVING COUNT(Emp_ID) > 5;
```

3. WHERE vs HAVING

Feature	WHERE Clause	HAVING Clause
Filters	Individual rows	Groups (after GROUP BY)
When Executed	Before GROUP BY	After GROUP BY
Aggregate Functions	❌ Cannot use	✅ Can use
Example	<code>WHERE Salary > 30000</code>	<code>HAVING AVG(Salary) > 50000</code>

4. SQL Query Execution Order

Understanding the strict order of execution is critical:

- ① FROM → Identify the table
- ② WHERE → Filter individual rows
- ③ GROUP BY → Group the remaining rows
- ④ HAVING → Filter the groups
- ⑤ SELECT → Select the output columns
- ⑥ ORDER BY → Sort the final result

Complex Example:

```
SELECT Department, AVG(Salary) AS Avg_Salary
FROM Employee
WHERE Status = 'Active'           -- ① Filter: only active employees
GROUP BY Department              -- ② Group by department
HAVING COUNT(Emp_ID) > 10        -- ③ Filter groups: dept with > 10 employees
    AND AVG(Salary) > 50000      --    and avg salary > 50,000
ORDER BY Avg_Salary DESC;        -- ④ Sort: highest avg salary first
```

★ Must-Write Points (for 10 marks)

1. **GROUP BY** groups rows with identical values and applies aggregate functions per group.
2. Without **GROUP BY**, aggregate functions return a single value for the entire table.
3. **HAVING** filters **groups** after grouping — cannot be replaced by **WHERE**.
4. **WHERE** filters **individual rows** before grouping; **HAVING** filters **groups** after.
5. **WHERE** cannot use aggregate functions; **HAVING** can and usually does.
6. Execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.
7. **ORDER BY** sorts the final grouped result; can sort by aggregate values (e.g., **ORDER BY AVG(Salary) DESC**).

⚡ Quick Recall

GROUP BY (group rows) → WHERE (filter rows BEFORE) → HAVING (filter groups AFTER, uses aggregates) → ORDER BY (sort final result) → Execution: FROM→WHERE→GROUP BY→HAVING→SELECT→ORDER BY